Task 6 – Design Challenge: Campus Event Registration Desktop Application (13 Marks, K5)

Run each program yourself and attach your own screenshots, each linked to the requirement or test it shows. Items marked `<fill>` need your own observations or measurements.

Stage 1 – Interpret the problem (2 marks)

Users: (1) student/volunteer who registers for an event; (2) committee member who views, filters and exports registrations.

Screens: Register form; Registrations table (with event filter and paging); dialogs (error, confirmation, file chooser for export); menu bar for navigation.

Data: `events(event_id, name, event_date)`, `registrations(reg_id, student_id, student_name, email, event_id, created_at)` in MySQL. A student can register for a given event at most once.

Rules and testable requirements

ID	Requirement (testable)
R1	Student ID must be 5–20 letters/digits. An empty or bad ID shows an error message and nothing is saved.
R2	Name must not be blank; email must match an email pattern; an event must be selected.
R3	A second registration with the same student ID and event is rejected with the message " is already registered for ". The table still has exactly one such row.
R4	While data loads or saves, the window still responds (drag the window, press other buttons; no freezing over 0.5 s).
R5	The table loads at most 50 rows per page; Next/Prev change pages; filter by event shows only that event's rows.
R6	Export writes the rows shown in the table to a CSV file chosen by the user.
R7	Closing the window with unsaved

ID	Requirement (testable)
	form text asks for confirmation.

Stage 2 – Screens, menu bar and flow

```
Menu bar:  File [ Export CSV…  |  Exit ]      View [ Register  |  View
registrations ]

┌──────────── Register screen ────────────┐      ┌──────────
Registrations screen ──────────┐
│ Student ID  [________]                  │      │ Event: [All events
▼]  [Prev] Page 1 [Next]│
│ Name        [____________]              │      │
┌ID─────┬Name──┬Email──┬Event──┐
│ Email       [____________]              │      │ │      │      │
│       │  JTable   │
│ Event       [ Hackathon ▼ ]             │      │
└───────┴───────┴───────┴───────┘
│             [Register]                  │
└─────────────────────────────────────────┘
│ status: Registered successfully / error │
└─────────────────────────────────────────┘

Flow:
 Start → Register screen ──(View ▸ View registrations)──▶
Registrations screen
   Register ▸ valid ▸ saved ▸ success message (stay on form)
   Register ▸ invalid ▸ red error text            Register ▸ duplicate
▸ red duplicate message
   File ▸ Export CSV ▸ JFileChooser ▸ file written (or error dialog)
   Window X / File ▸ Exit ▸ [unsaved text? → confirm dialog] ▸ close
```

Screens are switched with a `CardLayout`.

Stage 3 – Critical design decisions

- **Input validation** happens in the service layer (`RegistrationService.validate`), called *before* any database access, and the same check is used by the UI to show errors early.
- **Duplicate checking** happens in two places: (1) a `UNIQUE (student_id, event_id)` constraint in the database, which is the real guarantee even if two machines register at the same moment; (2) the DAO catches `SQLIntegrityConstraintViolationException` and throws `DuplicateRegistrationException`, which the UI turns into a clear message. Checking only with a `SELECT` first would leave a race condition.
- **Database access** only in `RegistrationDao`, using `PreparedStatement` (no SQL injection) and try-with-resources (no leaked connections).
- **Keeping long-running work off the event-dispatch thread:** every query and insert runs in `SwingWorker.doInBackground()`; results are applied to the table or labels

in `done()`, which runs on the EDT. Buttons are disabled while saving to avoid double submits. Paging with `LIMIT/OFFSET` keeps each load small.

Stage 4 – Classes and handlers

Class diagram (layers)

```
UI layer          EventApp (JFrame, JMenuBar, CardLayout, JTable,
SwingWorker)
                     | calls
Service layer     RegistrationService   validate(), register(),
page(), events()
                     | calls                      throws
DuplicateRegistrationException
Data layer        RegistrationDao  (JDBC: PreparedStatement, try-with-
resources)
                     |
                  MySQL  (events, registrations with
UNIQUE(student_id,event_id))
Model             Registration (studentId, name, email, eventName)
```

Pseudocode – Register action

```
PROCEDURE onRegisterClicked()
    err ← service.validate(id, name, email, event)
    IF err ≠ null THEN showRed(err); RETURN
    disable Register button ; showGray("Saving...")
    RUN IN BACKGROUND (SwingWorker)
        TRY   service.register(id, name, email, event)          // →
dao.insert (PreparedStatement)
        CATCH duplicate  → result ← "ID already registered for event"
        CATCH SQLException → result ← "Error: " + message
    ON COMPLETION (EDT)
        IF success THEN showGreen("Registered successfully"); clear
text fields
        ELSE showRed(result)
        enable Register button
END PROCEDURE
```

Pseudocode – Filter action

```
PROCEDURE onFilterChanged()
    pageNo ← 0
    eventName ← IF selected is "All events" THEN null ELSE selected
    RUN IN BACKGROUND (SwingWorker)
        rows ← dao.page(eventName, PAGE_SIZE, pageNo × PAGE_SIZE)  //
SELECT … LIMIT ? OFFSET ?
    ON COMPLETION (EDT)
        table.clear() ; FOR EACH row IN rows DO table.addRow(row) ;
pageLabel ← pageNo + 1
END PROCEDURE
```

Stage 5 – Implementation and demonstration

How to run: install MySQL, run `schema.sql`, put the MySQL Connector/J jar on the classpath, set the URL, user and password in `EventApp.main`, then: `javac *.java && java -cp .:mysql-connector-j-8.x.x.jar EventApp` (on Windows use `;` instead of `:`). All Java files compile without the driver; the driver is only needed at run time.

schema.sql

```
CREATE DATABASE IF NOT EXISTS campus_events;
USE campus_events;

CREATE TABLE IF NOT EXISTS events (
    event_id   INT AUTO_INCREMENT PRIMARY KEY,
    name       VARCHAR(100) NOT NULL UNIQUE,
    event_date DATE NOT NULL
);

CREATE TABLE IF NOT EXISTS registrations (
    reg_id     INT AUTO_INCREMENT PRIMARY KEY,
    student_id VARCHAR(20)  NOT NULL,
    student_name VARCHAR(100) NOT NULL,
    email      VARCHAR(100) NOT NULL,
    event_id   INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (event_id) REFERENCES events(event_id),
    UNIQUE KEY uq_student_event (student_id, event_id)   -- database-level duplicate guard
);

INSERT IGNORE INTO events(name, event_date) VALUES
 ('Cultural Night', '2026-12-05'), ('Hackathon', '2026-12-12'),
('Music Fest', '2026-12-20');
```

Registration.java and DuplicateRegistrationException.java

```
public class Registration {
    public final String studentId, name, email, eventName;
    public Registration(String studentId, String name, String email,
String eventName) {
        this.studentId = studentId; this.name = name; this.email =
email; this.eventName = eventName;
    }
}

public class DuplicateRegistrationException extends Exception {
    public DuplicateRegistrationException(String m) { super(m); }
}
```

RegistrationDao.java (data layer)

```
import java.sql.*;
import java.util.*;
```

```
/** Data-access layer: the only class that knows SQL/JDBC. */
public class RegistrationDao {
    private final String url, user, pwd;
    public RegistrationDao(String url, String user, String pwd) {
this.url = url; this.user = user; this.pwd = pwd; }

    private Connection open() throws SQLException { return
DriverManager.getConnection(url, user, pwd); }

    public List<String> eventNames() throws SQLException {
        List<String> out = new ArrayList<>();
        try (Connection c = open(); Statement s = c.createStatement();
             ResultSet rs = s.executeQuery("SELECT name FROM events
ORDER BY name")) {
            while (rs.next()) out.add(rs.getString(1));
        }
        return out;
    }

    public void insert(Registration r) throws SQLException,
DuplicateRegistrationException {
        String sql = "INSERT INTO registrations(student_id,
student_name, email, event_id) " +
                     "SELECT ?, ?, ?, event_id FROM events WHERE name
= ?";
        try (Connection c = open(); PreparedStatement ps =
c.prepareStatement(sql)) {
            ps.setString(1, r.studentId); ps.setString(2, r.name);
            ps.setString(3, r.email);     ps.setString(4,
r.eventName);
            ps.executeUpdate();
        } catch (SQLIntegrityConstraintViolationException dup) {
// MySQL error 1062
            throw new DuplicateRegistrationException(r.studentId + "
is already registered for " + r.eventName);
        }
    }

    /** One page of rows; eventName == null means all events. */
    public List<Object[]> page(String eventName, int limit, int
offset) throws SQLException {
        String sql = "SELECT r.student_id, r.student_name, r.email,
e.name FROM registrations r " +
                     "JOIN events e ON e.event_id = r.event_id " +
                     (eventName == null ? "" : "WHERE e.name = ? ") +
                     "ORDER BY r.reg_id LIMIT ? OFFSET ?";
        List<Object[]> rows = new ArrayList<>();
        try (Connection c = open(); PreparedStatement ps =
c.prepareStatement(sql)) {
```

```
            int i = 1;
            if (eventName != null) ps.setString(i++, eventName);
            ps.setInt(i++, limit); ps.setInt(i, offset);
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) rows.add(new Object[]
{rs.getString(1), rs.getString(2), rs.getString(3), rs.getString(4)});
            }
        }
        return rows;
    }
}
```

RegistrationService.java (service layer)

```
import java.sql.SQLException;
import java.util.List;
import java.util.regex.Pattern;

/** Service layer: validation + rules. No Swing, no SQL. */
public class RegistrationService {
    private static final Pattern EMAIL = Pattern.compile("^[\\w.+-]
+@[\\w-]+(\\.[\\w-]+)+$");
    private final RegistrationDao dao;
    public RegistrationService(RegistrationDao dao) { this.dao = dao;
}

    public String validate(String id, String name, String email,
Object event) {
        if (id == null || id.isBlank()) return "Student ID is
required";
        if (!id.trim().matches("[A-Za-z0-9]{5,20}")) return "Student
ID must be 5-20 letters/digits";
        if (name == null || name.isBlank()) return "Name is required";
        if (email == null || !EMAIL.matcher(email.trim()).matches())
return "Email is not valid";
        if (event == null) return "Select an event";
        return null;
    }

    public void register(String id, String name, String email, String
event)
            throws SQLException, DuplicateRegistrationException {
        String err = validate(id, name, email, event);
        if (err != null) throw new IllegalArgumentException(err);
        dao.insert(new Registration(id.trim(), name.trim(),
email.trim(), event));
    }

    public List<Object[]> page(String event, int pageSize, int pageNo)
throws SQLException {
        return dao.page(event, pageSize, pageNo * pageSize);
```

```
    }
    public List<String> events() throws SQLException { return
dao.eventNames(); }
}
```

EventApp.java (UI layer)

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.io.*;
import java.util.List;

/** UI layer: Swing only. Long DB work runs in SwingWorker, never on
the EDT. */
public class EventApp extends JFrame {
    private static final int PAGE_SIZE = 50;
    private final RegistrationService service;
    private final CardLayout cards = new CardLayout();
    private final JPanel deck = new JPanel(cards);

    // register screen
    private final JTextField id = new JTextField(12), name = new
JTextField(20), email = new JTextField(20);
    private final JComboBox<String> eventBox = new JComboBox<>();
    private final JLabel formStatus = new JLabel(" ");

    // view screen
    private final DefaultTableModel model =
            new DefaultTableModel(new Object[]{"Student ID", "Name",
"Email", "Event"}, 0) {
                @Override public boolean isCellEditable(int r, int c)
{ return false; } };
    private final JComboBox<String> filterBox = new JComboBox<>();
    private final JLabel pageLabel = new JLabel("Page 1");
    private int pageNo = 0;

    public EventApp(RegistrationService service) {
        super("Campus Event Registration");
        this.service = service;
        buildMenu();
        deck.add(registerPanel(), "register");
        deck.add(viewPanel(), "view");
        add(deck);
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        addWindowListener(new java.awt.event.WindowAdapter() {
            @Override public void
windowClosing(java.awt.event.WindowEvent e) {
                boolean dirty = !id.getText().isBlank() || !
name.getText().isBlank() || !email.getText().isBlank();
                if (!dirty ||
```

```
JOptionPane.showConfirmDialog(EventApp.this,
                        "Discard the unsaved registration and exit?",
"Exit", JOptionPane.YES_NO_OPTION)
                        == JOptionPane.YES_OPTION) { dispose();
System.exit(0); }
            }
        });
        setSize(640, 420);
        loadEvents();
    }

    private void buildMenu() {
        JMenuBar bar = new JMenuBar();
        JMenu file = new JMenu("File"), view = new JMenu("View");
        JMenuItem export = new JMenuItem("Export CSV..."), exit = new
JMenuItem("Exit");
        JMenuItem reg = new JMenuItem("Register"), list = new
JMenuItem("View registrations");
        export.addActionListener(e -> exportCsv());
        exit.addActionListener(e -> dispatchEvent(new
java.awt.event.WindowEvent(this,
java.awt.event.WindowEvent.WINDOW_CLOSING)));
        reg.addActionListener(e -> cards.show(deck, "register"));
        list.addActionListener(e -> { cards.show(deck, "view"); pageNo
= 0; loadPage(); });
        file.add(export); file.addSeparator(); file.add(exit);
        view.add(reg); view.add(list);
        bar.add(file); bar.add(view);
        setJMenuBar(bar);
    }

    private JPanel registerPanel() {
        JPanel p = new JPanel(new GridBagLayout());
        GridBagConstraints g = new GridBagConstraints();
        g.insets = new Insets(5, 5, 5, 5); g.anchor =
GridBagConstraints.WEST;
        String[] labels = {"Student ID", "Name", "Email", "Event"};
        Component[] fields = {id, name, email, eventBox};
        for (int i = 0; i < 4; i++) {
            g.gridx = 0; g.gridy = i; p.add(new JLabel(labels[i]), g);
            g.gridx = 1; p.add(fields[i], g);
        }
        JButton submit = new JButton("Register");
        g.gridx = 1; g.gridy = 4; p.add(submit, g);
        g.gridy = 5; p.add(formStatus, g);
        submit.addActionListener(e -> submitForm(submit));
        return p;
    }

    private void submitForm(JButton submit) {
```

```
        String err = service.validate(id.getText(), name.getText(),
email.getText(), eventBox.getSelectedItem());
        if (err != null) { formStatus.setForeground(Color.RED);
formStatus.setText(err); return; }
        submit.setEnabled(false);
formStatus.setForeground(Color.DARK_GRAY);
formStatus.setText("Saving...");
        new SwingWorker<Void, Void>() {
            @Override protected Void doInBackground() throws Exception
{
                service.register(id.getText(), name.getText(),
email.getText(), (String) eventBox.getSelectedItem());
                return null;
            }
            @Override protected void done() {
                submit.setEnabled(true);
                try {
                    get();
                    formStatus.setForeground(new Color(0, 120, 0));
formStatus.setText("Registered successfully");
                    id.setText(""); name.setText("");
email.setText("");
                } catch (Exception ex) {
                    Throwable t = ex.getCause() != null ?
ex.getCause() : ex;
                    formStatus.setForeground(Color.RED);
                    formStatus.setText(t instanceof
DuplicateRegistrationException ? t.getMessage() : "Error: " +
t.getMessage());
                }
            }
        }.execute();
    }

    private JPanel viewPanel() {
        JPanel p = new JPanel(new BorderLayout());
        JPanel top = new JPanel(new FlowLayout(FlowLayout.LEFT));
        top.add(new JLabel("Event:")); top.add(filterBox);
        JButton prev = new JButton("Prev"), next = new
JButton("Next");
        top.add(prev); top.add(pageLabel); top.add(next);
        p.add(top, BorderLayout.NORTH);
        p.add(new JScrollPane(new JTable(model)),
BorderLayout.CENTER);
        filterBox.addActionListener(e -> { pageNo = 0; loadPage(); });
        prev.addActionListener(e -> { if (pageNo > 0) { pageNo--;
loadPage(); } });
        next.addActionListener(e -> { pageNo++; loadPage(); });
        return p;
    }
```

```
    private void loadEvents() {
        new SwingWorker<List<String>, Void>() {
            @Override protected List<String> doInBackground() throws
Exception { return service.events(); }
            @Override protected void done() {
                try {
                    filterBox.addItem("All events");
                    for (String s : get()) { eventBox.addItem(s);
filterBox.addItem(s); }
                } catch (Exception ex) { formStatus.setText("Cannot
load events: " + ex.getMessage()); }
            }
        }.execute();
    }

    private void loadPage() {                       // paged:
LIMIT/OFFSET in a worker thread
        String sel = (String) filterBox.getSelectedItem();
        String ev = (sel == null || sel.equals("All events")) ? null :
sel;
        final int pn = pageNo;
        new SwingWorker<List<Object[]>, Void>() {
            @Override protected List<Object[]> doInBackground() throws
Exception { return service.page(ev, PAGE_SIZE, pn); }
            @Override protected void done() {
                try {
                    List<Object[]> rows = get();
                    if (rows.isEmpty() && pn > 0) { pageNo--; return;
}          // no next page
                    model.setRowCount(0);
                    for (Object[] r : rows) model.addRow(r);
                    pageLabel.setText("Page " + (pn + 1));
                } catch (Exception ex) {
JOptionPane.showMessageDialog(EventApp.this, "Load failed: " +
ex.getMessage()); }
            }
        }.execute();
    }

    private void exportCsv() {
        JFileChooser fc = new JFileChooser();
        if (fc.showSaveDialog(this) != JFileChooser.APPROVE_OPTION)
return;
        File f = fc.getSelectedFile();
        try (PrintWriter w = new PrintWriter(new FileWriter(f))) {
            w.println("student_id,name,email,event");
            for (int r = 0; r < model.getRowCount(); r++)
                w.println(model.getValueAt(r, 0) + "," +
model.getValueAt(r, 1) + "," + model.getValueAt(r, 2) + "," +
```

```
model.getValueAt(r, 3));
        } catch (IOException ex) { JOptionPane.showMessageDialog(this,
"Export failed: " + ex.getMessage()); }
    }

    public static void main(String[] args) {
        RegistrationDao dao = new
RegistrationDao("jdbc:mysql://localhost:3306/campus_events", "root",
"password");
        SwingUtilities.invokeLater(() -> new EventApp(new
RegistrationService(dao)).setVisible(true));
    }
}
```

Test cases to demonstrate (take a screenshot of each)

#	Test	Expected result
1	Valid registration: `CS12345`, Asha, `asha@mail.com`, Hackathon	"Registered successfully"; the row appears in the table
2	Empty student ID	"Student ID is required"; nothing saved
3	Invalid email `asha@`	"Email is not valid"
4	Register `CS12345` for Hackathon again	"CS12345 is already registered for Hackathon"; the table still has one row
5	Same student, different event	Succeeds (the rule is per event)
6	Filter by "Hackathon"	Only Hackathon rows are listed
7	Close the window with text typed in the form	Confirmation dialog appears

The validation rules (tests 2 and 3, and the empty-event case) were checked with a small test of `RegistrationService.validate` and returned: "Student ID is required", "Email is not valid", "Select an event", and `null` for a valid input. Database tests (1, 4–6) need your MySQL setup, so run those yourself `<fill>`.

Alternative concept and complexity

Load all registrations into the JTable at once, instead of one page at a time with `LIMIT/OFFSET` in a `SwingWorker`:

```
List<Object[]> all = dao.page(null, Integer.MAX_VALUE, 0);   // SELECT
… (no LIMIT)
```

```
for (Object[] r : all) model.addRow(r);                    //
100,000 rows into the model
```

- **Loading time, N = 100,000 rows:** loading everything is O(N) before the user sees anything (transfer, object creation and 100,000 `addRow` events). Paged loading costs O(P) per page with P = 50, so the first screen appears almost immediately; `OFFSET` makes later pages cost O(offset + P) in MySQL, so keyset paging (`WHERE reg_id > ? LIMIT ?`) is better for very deep pages.
- **Memory:** all-at-once holds N rows in memory: O(N) (for 100,000 rows with four strings each, tens of MB). Paged loading holds only P rows, O(P), independent of N.
- **Responsiveness:** loading everything on the event-dispatch thread would freeze the window for the whole load. Even inside a `SwingWorker`, filling the model with 100,000 rows in `done()` (or with many `publish` calls) keeps the EDT busy and makes scrolling and sorting sluggish. Paging with a worker keeps every EDT task small (50 rows), so the window stays responsive.
- **Own measurement <fill>:** insert 100,000 test rows, record the time to first display and the heap size (`Runtime.getRuntime().totalMemory() - freeMemory()`) for both approaches.